

ASSIGNMENT-6.5

AI-ASSISTED CODING

Name:E.Manu Sree

HTNO:2303A51324

BTNO:20

TASK:1

Use an AI tool to generate eligibility logic.

#prompt:

Generate Python code to check voting eligibility based on age and
Citizenship.

Code:

```
#Task 1:
#Generate Python code to check voting eligibility based on age and citizenship.
def check_voting_eligibility(age, is_citizen):
    if age >= 18 and is_citizen:
        return "Eligible to vote"
    else:
        return "Not eligible to vote"
# Example usage
age = 20
is_citizen = True
print(check_voting_eligibility(age, is_citizen)) # Output: Eligible to vote
age = 16
is_citizen = True
print(check_voting_eligibility(age, is_citizen)) # Output: Not eligible to vote
age = 25
is_citizen = False
print(check_voting_eligibility(age, is_citizen)) # Output: Not eligible to vote
age = 30
is_citizen = True
print(check_voting_eligibility(age, is_citizen)) # Output: Eligible to vote
```

Output:

```
Eligible to vote
Not eligible to vote
Not eligible to vote
Eligible to vote
```

Observation:

The AI-generated voting eligibility program determines eligibility based solely on age and citizenship. While the logic correctly enforces the minimum age requirement and citizenship condition, it lacks input validation and does not handle invalid or unexpected user inputs. Additionally, the system does not account for exceptional cases or regional voting rules, limiting its reliability and suitability for real-world.

Task:2

Use an AI tool to process strings using loops.

Prompt:

Generate Python code to count vowels and consonants in a string using a loop.

Code:

```
#Task 2:
#Use an AI tool to process strings using loops.
#Generate Python code to count vowels and consonants in a string using a loop
def count_vowels_consonants(input_string):
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0

    for char in input_string:
        if char.isalpha(): # Check if the character is a letter
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count
# Example usage
input_string = "Hello World"
vowels, consonants = count_vowels_consonants(input_string)
print(f"Vowels: {vowels}, Consonants: {consonants}") # Output: Vowels: 3, Consonants: 7
input_string = "Python Programming"
vowels, consonants = count_vowels_consonants(input_string)
print(f"Vowels: {vowels}, Consonants: {consonants}") # Output: Vowels: 4, Consonants: 14
```

Output:

```
Vowels: 3, Consonants: 7
Vowels: 4, Consonants: 13
```

Observation:

The AI-generated program correctly counts the number of vowels and consonants in a given string using a loop. It converts the input to lowercase to ensure case-insensitive comparison and uses `isalpha()` to ignore non-alphabetic characters. However, the program does not handle special cases such as empty input or accented characters, which may limit its accuracy in broader real-world scenario.

Task 3:

Use an AI tool to generate a complete program using classes, loops, and conditionals.

Prompt:

Generate a Python program for a library management system using classes, loops, and conditional statements.

Code:

```
#Task3:
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_available = True

    def borrow_book(self):
        if self.is_available:
            self.is_available = False
            return f"You have borrowed '{self.title}' by {self.author}."
        else:
            return f"Sorry, '{self.title}' is currently not available."

    def return_book(self):
        self.is_available = True
        return f"You have returned '{self.title}'. Thank you!"

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        return f"'{book.title}' by {book.author} has been added to the library."

    def display_books(self):
        for book in self.books:
            status = "Available" if book.is_available else "Not Available"
            print(f"Title: {book.title}, Author: {book.author}, Status: {status}")

# Example usage
library = Library()
book1 = Book("1984", "George Orwell")
book2 = Book("To Kill a Mockingbird", "Harper Lee")
print(library.add_book(book1))
print(library.add_book(book2))
library.display_books()
print(book1.borrow_book())
```

```
library.display_books()
print(book1.borrow_book())
library.display_books()
print(book1.return_book())
library.display_books()

# Reflection on AI-assisted coding experience:
# The AI-generated code for the library management system was well-structured and easy to understand.
# It effectively utilized classes to represent books and the library, and incorporated loops and conditionals to manage book availability.
# The suggestions provided by the AI were relevant and aligned with standard programming practices, making the coding process smoother and more ef
```

Output:

```
'1984' by George Orwell has been added to the library.  
'To Kill a Mockingbird' by Harper Lee has been added to the library.  
Title: 1984, Author: George Orwell, Status: Available  
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available  
You have borrowed '1984' by George Orwell.  
Title: 1984, Author: George Orwell, Status: Not Available  
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available  
You have returned '1984'. Thank you!  
Title: 1984, Author: George Orwell, Status: Available  
Title: To Kill a Mockingbird, Author: Harper Lee, Status: Available
```

Observation:

The AI-generated library management system effectively demonstrates object-oriented programming by using separate Book and Library classes to manage books and their availability. It provides basic functionalities such as adding books, displaying available books, borrowing, and returning books through a menu-driven interface. However, the system lacks input validation, does not handle duplicate book titles, and does not persist data between runs, which limits its scalability and suitability for real-world library applications.

Task 4:

Use an AI tool to generate an attendance management class.

Prompt:

Generate a Python class to mark and display student attendance using loops.

Code:

```
#Task 4:  
#Generate a Python class to mark and display student attendance using loops.  
class Student:  
    def __init__(self, name):  
        self.name = name  
        self.attendance_record = []  
  
    def mark_attendance(self, status):  
        if status in ['Present', 'Absent']:  
            self.attendance_record.append(status)  
        else:  
            print("Invalid status. Please use 'Present' or 'Absent'.")  
  
    def display_attendance(self):  
        print(f"Attendance record for {self.name}:")  
        for day, status in enumerate(self.attendance_record, start=1):  
            print(f"Day {day}: {status}")  
  
# Example usage  
student = Student("Alice")  
student.mark_attendance("Present")  
student.mark_attendance("Absent")  
student.mark_attendance("Present")  
student.display_attendance() # Output: Attendance record for Alice: Day 1: Present Day 2: Absent Day 3: Present  
student.mark_attendance("Late") # Output: Invalid status. Please use 'Present' or 'Absent'.
```

Output:

```
Attendance record for Alice:  
Day 1: Present  
Day 2: Absent  
Day 3: Present  
Invalid status. Please use 'Present' or 'Absent'.
```

Observation:

The AI-generated student attendance system uses an object-oriented approach to record and track attendance for a student. It allows attendance to be marked day by day with basic input validation and accurately calculates the number of days present out of the total days. However, the system is limited to a single student, does not store attendance records persistently, and lacks support for attendance analysis such as percentages or multiple students, reducing its applicability in real-world academic environments.

Task:5

Use an AI tool to complete a navigation menu.

Prompt:

Generate a Python program using loops and conditionals
to simulate an ATM menu.

Code:

```

#Task 5:
#Generate a Python program using loops and conditionals to simulate an ATM menu."
def atm_menu():
    balance = 1000 # Initial balance
    while True:
        print("\nATM Menu:")
        print("1. Check Balance")
        print("2. Deposit Money")
        print("3. Withdraw Money")
        print("4. Exit")
        choice = input("Please select an option (1-4): ")

        if choice == '1':
            print(f"Your current balance is: ${balance}")
        elif choice == '2':
            amount = float(input("Enter amount to deposit: $"))
            if amount > 0:
                balance += amount
                print(f"${amount} deposited successfully.")
            else:
                print("Invalid amount. Please enter a positive value.")
        elif choice == '3':
            amount = float(input("Enter amount to withdraw: $"))
            if 0 < amount <= balance:
                balance -= amount
                print(f"${amount} withdrawn successfully.")
            else:
                print("Invalid amount. Please enter a positive value within your balance.")
        elif choice == '4':
            print("Thank you for using the ATM. Goodbye!")
            break
        else:
            print("Invalid selection. Please choose a valid option.")

# Example usage
atm_menu()

```

Output:

```
ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 1
Your current balance is: $1000

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 2
Enter amount to deposit: $1000
$1000.0 deposited successfully.

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 3
Enter amount to withdraw: $500
$500.0 withdrawn successfully.

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 4
Thank you for using the ATM. Goodbye!
```

Observation:

The AI-generated ATM simulation program provides basic banking operations such as checking balance, depositing money, and withdrawing money through a menu-driven interface. It correctly updates the account balance and includes simple validation for deposit and withdrawal amounts. However, the system lacks user authentication, exception handling for invalid numeric inputs, transaction history, and data persistence, making it unsuitable for real-world banking applications.